Group members

Duku Henry Mugume	**2600726521**	**26/E/26521/PS**
Mayen Ayiik Mayen	**2600726534**	**26/E/26534/PS**

GitHub link: https://github.com/dukuhenry/Simple-Banking-System

Title: Simple Banking System - CSC 1102 Java Project

Description: A beginner Java Swing desktop application that allows a user to create bank accounts, deposit, withdraw, transfer money, check balances, and view transaction history. No database, no login, no networking. Data is stored in arrays in memory while the program runs. Built using Object-Oriented Programming principles: encapsulation, composition, and separation of duties.

Classes and Purpose

Class	Purpose	Key Attributes / Methods
Customer.java	Stores person details only	customerId, fullName, phone , address / Getters, Setters, toString( )
Account.java	Stores money + owner, guards balance	accountNumber, customer, balance, accountType / deposit(), withdraw(), getBalance()
Transaction.java	Stores one receipt / history	transactionId , accountNumber, type , amount, date / Gette rs only, no setters
Bank.java	Manager, holds arrays, finds accounts	accounts[100] , transactions[1000] , accountCoun t / addAccount(), findAccount() , deposit(), withdraw(), transfer(), view AllAccounts()
BankSystemGUI.java	Window, buttons, user input	JFrame, JPanel, JTextField, JButton, JTextArea, JOptionPane ActionListener / for each button, main()

How Classes Interact

The GUI does not touch data directly. When a user clicks a button, the GUI reads the text fields and calls a method in Bank. Bank uses findAccount() to search its array with a for loop. If found, Bank asks the Account to deposit() or withdraw(). Account checks the amount with if/else and updates its private balance. If successful, Bank creates a new Transaction object as a receipt and stores it in its transaction array. The GUI then shows a popup or updates the text area.

How To Run + Test Results

To Run: Open the folder in VS Code with Extension Pack for Java. Open BankSystemGUI.java and click Run, or in terminal run javac *.java then java BankSystemGUI

TEST 1 - Create Account

Type: Acc No=101, Name=Duku Henry, Phone=0770000000, Address=Kampala, Amount=1000, Type=Savings

Click: Create Account

Expect: Popup "Account created successfully!" + View shows Acc 101, Balance 1000

Click: Clear

TEST 2 - Deposit

Type: Acc No=101, Amount=500

Click: Deposit

Expect: Popup "Deposit successful!"

Click: Check Balance , Expect: 1500

Click: Clear

TEST 3 - Withdraw

Type: Acc No=101, Amount=200

Click: Withdraw

Expect: Popup "Withdraw successful!"

Click: Check Balance , Expect: 1300

Click: Clear

TEST 4 - Withdraw Too Much (Bad Input Test)

Type: Acc No=101, Amount=5000

Click: Withdraw

Expect: Popup "Withdraw failed!" + Balance still 1300

Click: Clear

TEST 5 - Create Second Account + Transfer

Type: Acc No=102, Name=Mayen Ayiik, Phone=0780000000, Address=Mukono, Amount=0, Type= Click: Create Account

Click: Clear

Type: Acc No=101, To Account=102, Amount=300

Click: Transfer

Expect: Popup "Transfer successful!"

Check Balance 101 , Expect: 1000

Check Balance 102 , Expect: 300

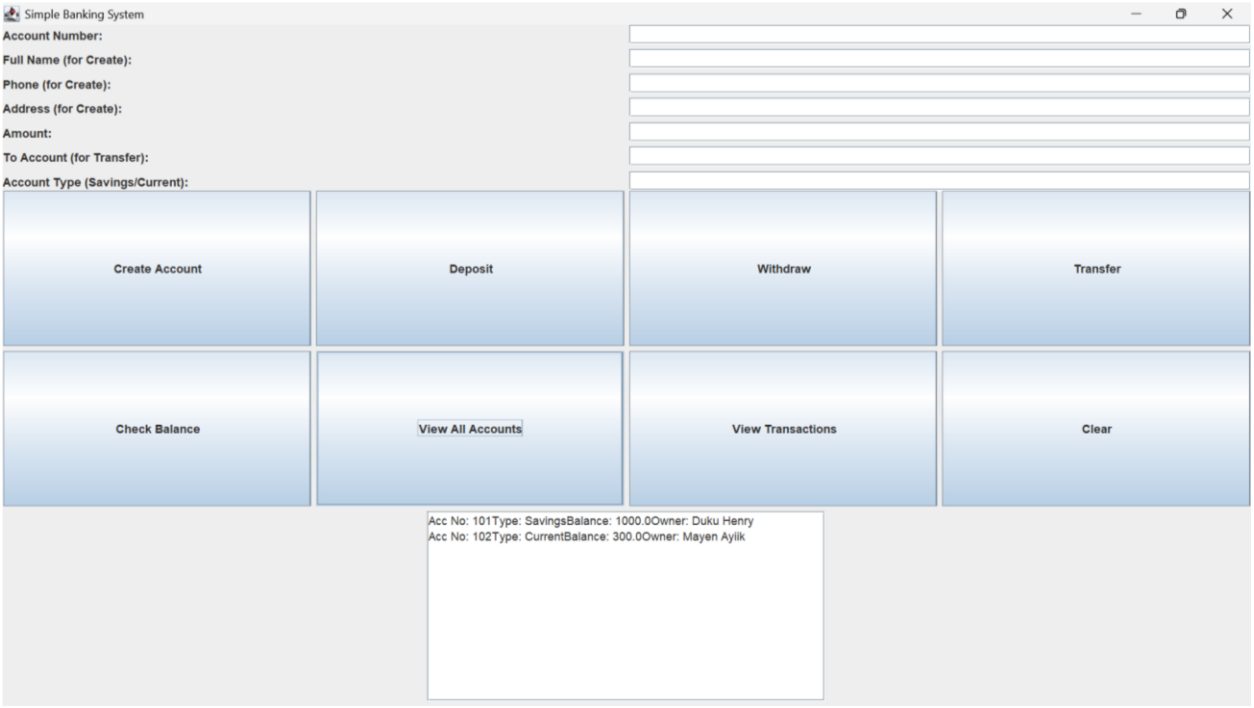
Click: Clear

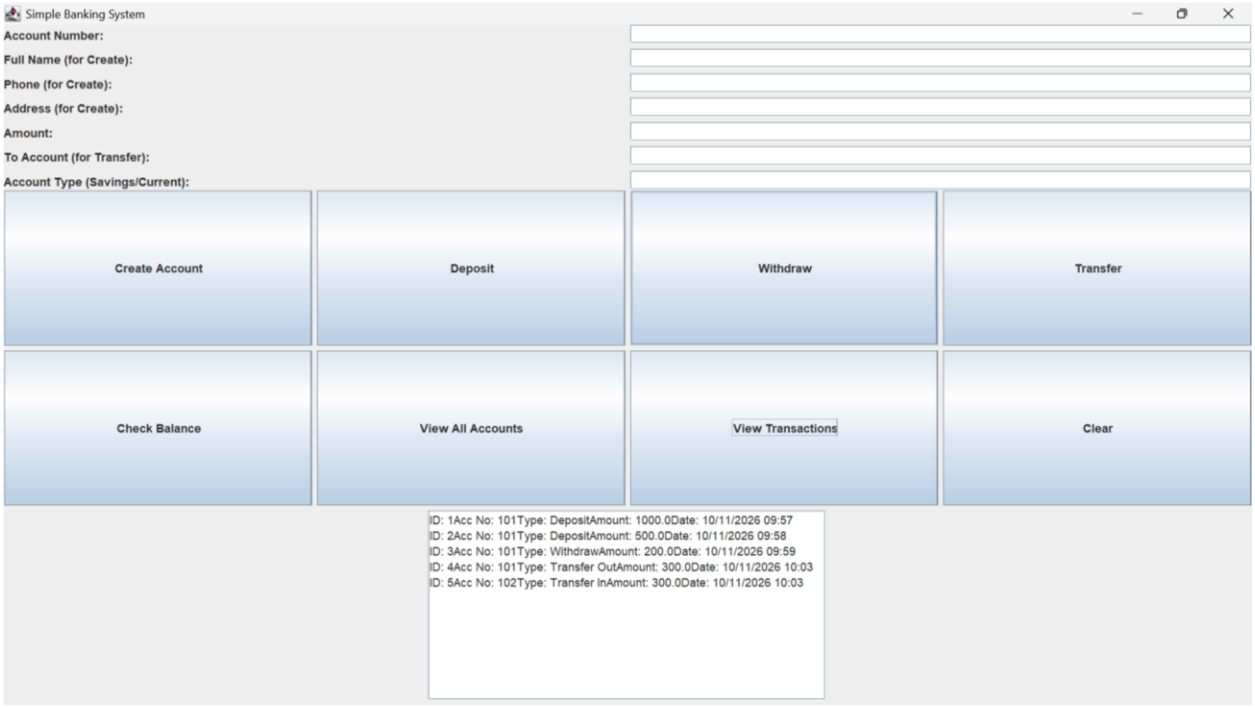
TEST 6 - View All + View Transactions

Click: View All Accounts , Expect: Both accounts listed in text area

Leave Acc No empty, Click: View Transactions , Expect: All receipts listed

Type: Acc No=101, Click: View Transactions , Expect: Only 101's receipts





Challenges + Conclusion

Challenges:

1) Learning why balance must be private and only changed through methods.
2) Understanding accountCount and why loops must stop at accountCount not 100 to avoid empty boxes.
3) Handling NumberFormatException so the program does not crash.

Conclusion:
This project taught me OOP in practice: encapsulation protects data, composition lets Account hold a Customer, arrays + loops manage many objects, and Swing lets a GUI talk to logic through one Bank object. The system meets all requirements: Swing only, arrays only, no database, no login.